

OptArrow Software Specification

Introduction

OptArrow is an optimization integration engine designed to facilitate communication between optimization clients and solver backends. This document provides a detailed specification focusing on the mechanisms, interfaces, and data transformations within OptArrow. The specification ensures that all descriptions align with the verified claims from prior documentation, emphasizing the functional aspects observed in practice.

1. System Architecture

1.1 Data Interchange with Apache Arrow

Mechanism

OptArrow utilizes Apache Arrow for efficient cross-language data interchange, primarily due to Arrow's capabilities for zero-copy data exchange. This allows seamless interoperability between Python and Julia environments, minimizing overhead.

Data Transformation

- **Input:** Python Dictionary representing optimization problem details.
- **Transformation:** Python data structures are converted to Arrow's in-memory columnar format.
- **Output:** Arrow IPC (Inter-Process Communication) format, enabling direct zero-copy access from Julia.

1.2 Solver Integration: HiGHS Library

Mechanism

OptArrow integrates the HiGHS library within its dedicated solver layer, specifically targeting Linear Programming (LP) and Quadratic Programming (QP).

Interface

- **Input:** Arrow IPC data containing problem definitions.
- **Interface Layer:** Transforms Arrow IPC to the native HiGHS library format, leveraging HiGHS APIs to execute optimization tasks.
- **Output:** Optimization results transform back from HiGHS specific structures to Arrow IPC.

1.3 Task Execution Modes

Synchronous and Asynchronous Execution

OptArrow supports both synchronous and asynchronous task execution, providing flexibility

for diverse computational requirements.

- **Synchronous Execution:** Immediate processing where the client waits for completion. Data flows directly from Arrow IPC to HiGHS and results back to client-native structures.
- **Asynchronous Execution:** Utilizes a queue system where tasks are deferred, allowing them to be processed when resources permit, and clients are notified upon completion.

2. Security Mechanisms

OptArrow ensures secure data handling through implemented encryption at rest and transit. This mechanism is crucial to protect sensitive optimization problem data and results.

Encryption and Secure Handling

- **Data at Rest:** Encrypted using industry-standard encryption algorithms.
- **Data in Transit:** Utilizes secure protocols for encryption, ensuring data integrity and confidentiality across different layers and interfaces.

3. Compliance and Architectural Considerations

3.1 ISO/IEC TS 33061:2021 Compliance

OptArrow's architecture aligns with ISO/IEC TS 33061:2021 standards, ensuring that regulatory adherence is integrated into its operations. This compliance is maintained through continuous auditing and process checks.

3.2 Scalability and Flexibility

Architectural Challenges

While designed for flexibility and scalability, OptArrow navigates administrative burdens through process optimizations, balancing between delivering performance and maintaining process rigor.

Solutions

- **Scalable Architecture:** Implements modular design principles allowing horizontal scaling across computation nodes.
- **Administrative Processes:** Streamlining workflows and utilizing automated processes to reduce overhead while ensuring compliance.

Conclusion

OptArrow provides an efficient platform for optimization that integrates robust data interchange mechanisms and dedicated solver support. With a focus on security, compliance, and flexibility, it is well-equipped to handle complex optimization scenarios across diverse operational environments.

This specification, built on validated claims, serves as a technical foundation for further development and refinement of OptArrow's capabilities, ensuring alignment with stakeholder

expectations and operational requirements.